

Математическое моделирование

Лабораторная работа № 2

Суннатилло Махмудов

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Условие задачи	10
4	Полярные траектории: катер (ODE) и лодка (аналитически)	11
4.1	Инициализация проекта и загрузка пакетов	11
4.2	Параметры задачи	12
4.3	Определение модели	12
4.4	Запуск 1: $r_0 = s/(n+1)$, $t \in (1e-9, 8)$	14
4.5	Запуск 2: $r_0 = s/(n-1)$, $t \in (1e-9, 15)$	14
5	Параметрическое исследование: катер (ODE) и лодка (аналитически) в полярных координатах	16
5.1	Активация проекта и загрузка пакетов	16
5.2	Определение модели	17
5.3	Определение параметров в Dict	17
5.4	Функция-обертка для запуска одного эксперимента	18
5.5	Запуск базового эксперимента (кэширование)	20
5.6	Визуализация базового эксперимента	21
5.7	Второй базовый эксперимент (как у тебя во 2-й части): <code>case=:minus</code> , <code>tmax=15</code>	22
5.8	Параметрическое сканирование	23
5.9	Запуск всех экспериментов и сбор результатов	24
5.10	Анализ и визуализация результатов сканирования	26
5.11	Бенчмаркинг с разными параметрами	28
5.12	Сохранение всех результатов	30
5.13	Анализ результатов моделирования	31
5.14	Базовые эксперименты	31
5.15	Параметрическое сканирование по n	33
5.16	Анализ метрики <code>scale_ratio</code>	34
5.17	Время вычислений	35
6	Выводы	37

Список иллюстраций

5.1	Базовый эксперимент (case=plus)	31
5.2	Базовый эксперимент (case=minus)	32
5.3	Сканирование траекторий катера	33
5.4	Зависимость scale_ratio от n	34
5.5	Зависимость времени вычисления от n	35

Список таблиц

1 Цель работы

Приведем один из примеров построения математических моделей для выбора правильной стратегии при решении задач поиска. Например, рассмотрим задачу преследования браконьеров береговой охраной. На море в тумане катер береговой охраны преследует лодку браконьеров. Через определенный промежуток времени туман рассеивается, и лодка обнаруживается на расстоянии k км от катера. Затем лодка снова скрывается в тумане и уходит прямолинейно в неизвестном направлении. Известно, что скорость катера в n раза больше скорости браконьерской лодки. Необходимо определить по какой траектории необходимо двигаться катеру, чтоб нагнать лодку.

2 Задание

1. Провести необходимые рассуждения и вывод дифференциальных уравнений, если скорость катера больше скорости лодки в n раз.
2. Построить траекторию движения катера и лодки для двух случаев.
3. Определить по графику точку пересечения катера и лодки.

3 Выполнение лабораторной работы

Принимаем за $t_0 = 0$, $X_0 = 0$ - место нахождения лодки браконьеров в момент обнаружения, $X_0 = k$ - место нахождения катера береговой охраны относительно лодки браконьеров в момент обнаружения лодки.

Введем полярные координаты. Считаем, что полюс - это точка обнаружения лодки браконьеров $x_0 = 0$ ($\theta = x_0 = 0$), а полярная ось r проходит через точку нахождения катера береговой охраны.

Чтобы найти расстояние x (расстояние после которого катер начнет двигаться вокруг полюса), необходимо составить простое уравнение. Пусть через время t катер и лодка окажутся на одном расстоянии x от полюса. За это время лодка пройдет x , а катер $x - k$ (или $x + k$, в зависимости от начального положения катера относительно полюса). Время, за которое они пройдут это расстояние, вычисляется как $\frac{x}{v}$ или $\frac{x+k}{v}$ (для второго случая $\frac{x-k}{v}$). Так как время одно и то же, то эти величины одинаковы. Тогда неизвестное расстояние можно найти из следующего уравнения: $\frac{x}{v} = \frac{x+k}{v}$ - в первом случае, $\frac{x}{v} = \frac{x-k}{v}$ во втором случае.

Отсюда мы найдем два значения x_1 и x_2 , задачу будем решать для двух случаев.

$$x_1 = \frac{k}{n+1}, \text{ при } \theta = 0$$

$$x_2 = \frac{k}{n-1}, \text{ при } \theta = -\pi$$

После того, как катер береговой охраны окажется на одном расстоянии от полюса, что и лодка, он должен сменить прямолинейную траекторию и начать двигаться вокруг полюса удаляясь от него со скоростью лодки v . Для этого скорость катера раскладываем на две составляющие: v_r - радиальная скорость и v_t - тангенциальная скорость. Радиальная скорость - это скорость, с которой катер удаляется от полюса

$v_r = \frac{dr}{dt}$. Нам нужно, чтобы эта скорость была равна скорости лодки, поэтому полагаем $v = \frac{dr}{dt}$. Тангенциальная скорость – это линейная скорость вращения катера относительно полюса. Она равна произведению угловой скорости $\frac{d\theta}{dt}$ на радиус r , $v_t = r \frac{d\theta}{dt}$. Найдем тангенциальную скорость для нашей задачи $v_t = r \frac{d\theta}{dt}$. Вектора образуют прямоугольный треугольник, откуда по теореме Пифагора можно найти тангенциальную скорость $v_t = \sqrt{n^2 v_r^2 - v^2}$. Поскольку, радиальная скорость равна v , то тангенциальную скорость находим из уравнения $v_t = \sqrt{n^2 v^2 - v^2}$. Следовательно, $v_r = v \sqrt{n^2 - 1}$.

Тогда получаем $r \frac{d\theta}{dt} = v \sqrt{n^2 - 1}$

Решение исходной задачи сводится к решению системы из двух дифференциальных уравнений

$$\begin{cases} \frac{dr}{dt} = v \\ r \frac{d\theta}{dt} = v \sqrt{n^2 - 1} \end{cases}$$

с начальными условиями

$$\begin{cases} \theta_0 = 0 \\ r_0 = \frac{k}{n+1} \end{cases}$$

$$\begin{cases} \theta_0 = -\pi \\ r_0 = \frac{k}{n-1} \end{cases}$$

Исключая из полученной системы производную по t , можно перейти к следующему уравнению: $\frac{dr}{d\theta} = \frac{r}{\sqrt{n^2 - 1}}$

Начальные условия остаются прежними. Решив это уравнение, мы получим траекторию движения катера в полярных координатах. Теперь, когда нам известно все, что нам нужно, построим траекторию движения катера и лодки для двух случаев.

3.1 Условие задачи

На море в тумане катер береговой охраны преследует лодку браконьеров. Через определенный промежуток времени туман рассеивается, и лодка обнаруживается на расстоянии 20 км от катера. Затем лодка снова скрывается в тумане и уходит прямолинейно в неизвестном направлении. Известно, что скорость катера в 5 раз больше скорости браконьерской лодки

Для моделирования процесса и построения графиков использовались внешние файлы с программным кодом:

4 Полярные траектории: катер (ODE) и лодка (аналитически)

Цель: построить траектории в полярных координатах для «катера» (решение ОДУ) и «лодки» (заданная прямая в декартовых координатах, переведённая в полярные).

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Параметры задачи

```
n = 5
s = 20
fi = 3/4 * pi

p = (n = n, s = s, fi = fi)
```

4.3 Определение модели

Катер: $dr/d\theta = r / \sqrt{n^2 - 1}$ Здесь независимая переменная — θ (мы используем t как θ , как принято в ODEProblem).

```
function cutter_ode!(dr, r, p, ⍅)
    dr[1] = r[1] / sqrt(p.n^2 - 1)
end
```

Лодка: линия $x(t)=t$, $y(t)=\tan(fi+\pi)*t$, затем перевод в (r, θ) В Julia правильнее использовать $\text{atan}(y, x)$, чтобы угол был в верном квадранте.

```
function boat_polar(t, p)
    k = tan(p.fi + pi)
    x = t
    y = k * t
    r = hypot(x, y)           # sqrt(x^2 + y^2)
    ⍅ = atan(y, x)
    return r, ⍅
end
```

Утилита: генерация траектории лодки для набора t

```

function make_boat_curve(tgrid, p)
    r = VectorFloat64(undef, length(tgrid))
    ϕ = VectorFloat64(undef, length(tgrid))
    for (i, tt) in pairs(tgrid)
        ri, ϕi = boat_polar(tt, p)
        r[i] = ri
        ϕ[i] = ϕi
    end
    return r, ϕ
end

```

Утилита: один прогон (и график, и таблицы, и сохранение)

```

function run_case(case_name; r0, ϕspan=(0.0, 2pi), nϕ=10_000, tmin=1

```

— Катер (ODE) —

```

    ϕgrid = collect(LinRange(ϕspan[1], ϕspan[2], nϕ))
    prob = ODEProblem(cutter_ode!, [r0], ϕspan, p)
    sol = solve(prob, Tsit5(), saveat=ϕgrid)

    df_cutter = DataFrame(ϕ = sol.t, r = first.(sol.u))

```

— Лодка (аналитически) —

```

    tgrid = collect(LinRange(tmin, tmax, nt))
    r_boat, ϕ_boat = make_boat_curve(tgrid, p)
    df_boat = DataFrame(t = tgrid, ϕ = ϕ_boat, r = r_boat)

```

— Визуализация —

```

plt = plot(sol, proj=:polar, label="XXXX", xlabel="ϕ", ylabel="r",
           title="XXXXXXXX XXXXXXXXXX - $case_name", lw=2, legend=:none)
plot!(plt, ϕ_boat, r_boat, proj=:polar, label="XXXX", lw=2)

```

— Сохранение —

```

savefig(plt, plotsdir(script_name, "polar_$case_name.png"))
@save datadir(script_name, "data_$case_name.jld2") df_cutter df_boat

return (plt=plt, df_cutter=df_cutter, df_boat=df_boat)
end

```

4.4 Запуск 1: $r_0 = s/(n+1)$, $t \in (1e-9, 8)$

```

case1_r0 = p.s / (p.n + 1)
res1 = run_case("r0=s_div_(n+1)"; r0=case1_r0, tmax=8.0, p=p)

println("XXXX 1 - XXXXXX 5 XXXXX (XXXX):")
println(first(res1.df_cutter, 5))
println("\nXXXX 1 - XXXXXX 5 XXXXX (XXXX):")
println(first(res1.df_boat, 5))

```

4.5 Запуск 2: $r_0 = s/(n-1)$, $t \in (1e-9, 15)$

```

case2_r0 = p.s / (p.n - 1)
res2 = run_case("r0=s_div_(n-1)"; r0=case2_r0, tmax=15.0, p=p)

```

```
println("\nСредняя скорость 2-го судна (км/ч):")
println(first(res2.df_cutter, 5))
println("\nСредняя скорость 5-го судна (км/ч):")
println(first(res2.df_boat, 5))
```

(опционально) показать последний график в интерактивной среде

```
res2.plt
```

5 Параметрическое исследование: катер (ODE) и лодка (аналитически) в полярных координатах

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

Установка каталогов

```
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.2 Определение модели

Катер: $dr/d\theta = r / \sqrt{n^2 - 1}$

```
function cutter_ode!(dr, r, p, ⓧ)
    dr[1] = r[1] / sqrt(p.n^2 - 1)
end
```

Лодка: $x=t, y=\tan(\theta+\pi)*t \rightarrow (r, \theta)$ Важно: используем `atan(y, x)`, чтобы угол был корректен по квадрантам

```
function boat_polar(t, p)
    k = tan(p.fi + pi)
    x = t
    y = k * t
    r = hypot(x, y)
    ⓧ = atan(y, x)
    return r, ⓧ
end
```

5.3 Определение параметров в Dict

Все параметры — в одном Dict, как в шаблоне. Здесь «case» управляет выбором `r0: :plus (s/(n+1))` или `:minus (s/(n-1))`.

```
base_params = Dict(
    :n => 5,
    :s => 20,
    :fi => 3/4*pi,
```

```

:case => :plus,                                # :plus 000 :minus
:span => (0.0, 2*pi),                            # 00000000 00 0
:n0 => 10_000,                                  # 00000 00000 000 0000000000 00000000

:tspan_boat => (1e-9, 8.0),                     # 00000000 00 t 000 00000
:nt_boat => 1_000,                             # 00000 00000 000 00000

:solver => Tsit5(),
:experiment_name => "base_experiment"
)

println("0000000 000000000 000000000000:")
for (key, value) in base_params
    println(" $key = $value")
end

```

5.4 Функция-обертка для запуска одного эксперимента

Возвращаем Dict со строковыми ключами (как в твоём шаблоне).

```

function run_single_experiment(params::Dict)
    @unpack n, s, fi, case, span, n0, tspan_boat, nt_boat, solver =

```

Параметры для ODE в виде именованного кортежа

```
p = (n=n, s=s, fi=fi)
```

Начальное условие r0 зависит от кейса

```
r0 = case == :plus ? s/(n+1) : s/(n-1)
```

— Катер (ODE) —

```
θgrid = collect(LinRange(θspan[1], θspan[2], nθ))  
prob = ODEProblem(cutter_ode!, [r0], θspan, p)  
sol = solve(prob, solver; saveat=θgrid)  
  
r_cutter = first.(sol.u)
```

— Лодка (аналитика) —

```
tgrid = collect(LinRange(tspan_boat[1], tspan_boat[2], nt_boat))  
r_boat = VectorFloat64(undef, length(tgrid))  
θ_boat = VectorFloat64(undef, length(tgrid))  
for (i, tt) in pairs(tgrid)  
    ri, θi = boat_polar(tt, p)  
    r_boat[i] = ri  
    θ_boat[i] = θi  
end
```

— Мини-анализ (несколько метрик, чтобы было что сравнивать в сканировании)

— 1) финальный радиус катера на $\theta=\text{end}$

```
r_cutter_final = r_cutter[end]
```

2) максимальный радиус лодки на её сетке

```
r_boat_max = maximum(r_boat)
```

3) простая «разница масштабов» (без строгого физического смысла, но для сравнения кейсов полезно)

```

scale_ratio = r_cutter_final / r_boat_max

return Dict(
    "solution" => sol,
    "t_points" => sol.t,
    "r_cutter" => r_cutter,

    "t_points_boat" => tgrid,
    "t_boat" => t_boat,
    "r_boat" => r_boat,

    "r0" => r0,
    "r_cutter_final" => r_cutter_final,
    "r_boat_max" => r_boat_max,
    "scale_ratio" => scale_ratio,

    "parameters" => params
)
end

```

5.5 Запуск базового эксперимента (кэширование)

```

data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "polar",
    tag = false,

```

```

        verbose = true
    )

println("\nXXXXXXXXXX XXXXXXXX XXXXXXXXXXXXXXX:")
println(" r0: ", data_base["r0"])
println(" r_cutter_final: ", data_base["r_cutter_final"])
println(" r_boat_max: ", data_base["r_boat_max"])
println(" scale_ratio: ", round(data_base["scale_ratio"]; digits=4))
println(" XXXX XXXXXXXXXXXXXXX: ", path_base)

```

5.6 Визуализация базового эксперимента

```

p1 = plot(
    data_base["_points"], data_base["r_cutter"],
    proj=:polar,
    label="XXXXX",
    xlabel="θ",
    ylabel="r",
    title="XXXXXXXXXX XXXXXXXXXXXXXXX (case=$(base_params[:case]))",
    lw=2,
    legend=:topleft,
    grid=true
)
plot!(
    p1,
    data_base["_boat"], data_base["r_boat"],
    proj=:polar,
    label="XXXXX",

```

```

        lw=2
    )

savefig(p1, plotsdir(script_name, "single_experiment.png"))

```

5.7 Второй базовый эксперимент (как у тебя во 2-й части): case=:minus, tmax=15

```

base_params2 = copy(base_params)
base_params2[:case] = :minus
base_params2[:tspan_boat] = (1e-9, 15.0)
base_params2[:experiment_name] = "base_experiment_minus"

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "polar",
    tag = false,
    verbose = true
)

p1b = plot(
    data_base2["_points"], data_base2["r_cutter"],
    proj=:polar,
    label="XXXXX",
    xlabel="X",

```

```

        ylabel="r",
        title="XXXXXXXX XXXXXXXXXXXX (case=$(base_params2[:case]))",
        lw=2,
        legend=:topleft,
        grid=true
    )
plot!(
    p1b,
    data_base2["_boat"], data_base2["r_boat"],
    proj=:polar,
    label="XXXXX",
    lw=2
)
savefig(p1b, plotsdir(script_name, "single_experiment_minus.png"))

```

5.8 Параметрическое сканирование

В твоей задаче естественно сканировать n (оно влияет и на ODE, и на r_0). При желании можно заменить на fi или s (или сделать сетку по двум параметрам).

```

param_grid = Dict(
    :n => [3, 4, 5, 6, 8, 10],          # XXXXXXXXXXXX n
    :s => [20],                        # XXXXXXXXXXXX
    :fi => [3/4*pi],                   # XXXXXXXXXXXX

    :case => [:plus, :minus],          # XXXXXXXXXXXX XXX XXXXXX
    :_span => [(0.0, 2*pi)],
    :n_ => [10_000],

```

```
:tspan_boat => [(1e-9, 8.0)],          # [ ] [ ] [ ],
:nt_boat    => [1_000],

:solver     => [Tsit5()],
:experiment_name => ["parametric_scan"]

)

all_params = dict_list(param_grid)

println("\n" * "="^60)
println("[ ] [ ]")
println("[ ] [ ] [ ]: ", length(all_params))
println("[ ] n: ", param_grid[:n])
println("[ ] case: ", param_grid[:case])
println("="^60)
```

5.9 Запуск всех экспериментов и сбор результатов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println("XXXXXXXX: $i/${length(all_params)} | n=${params[:n]} | c

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
```

```

    prefix = "scan",
    tag = false,
    verbose = false
)

```

Сводка по эксперименту

```

result_summary = merge(
  params,
  Dict(
    :r0 => data["r0"],
    :r_cutter_final => data["r_cutter_final"],
    :r_boat_max => data["r_boat_max"],
    :scale_ratio => data["scale_ratio"],
    :filepath => path
  )
)
push!(all_results, result_summary)

```

Полные данные (катер) — удобно для дальнейших графиков/анализа

```

df = DataFrame(
  ⬜ = data["⬜_points"],
  r = data["r_cutter"],
  n = fill(params[:n], length(data["⬜_points"])),
  case = fill(string(params[:case]), length(data["⬜_points"]))
)
push!(all_dfs, df)
end

```

5.10 Анализ и визуализация результатов сканирования

```
results_df = DataFrame(all_results)
println("\nXXXXXXXX XXXXXX XXXXXXXXXXXX (XXXXXX XXXXX):")
println(first(results_df, 10))
```

Сравнительный график траекторий катера для всех комбинаций (по θ)

```
p2 = plot(size=(900, 520), dpi=150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    plot!(
        p2,
        data["_points"], data["r_cutter"],
        label="n=$(params[:n]), case=$(params[:case])",
        lw=2,
        alpha=0.8
    )
end
plot!(
```

```

    p2,
    xlabel="n",
    ylabel="r(n)",
    title="XXXXXXXXXXXX: XXXXXXXX XXXXXX (ODE) XXX XXXXXX n X case",
    legend=:outerright,
    grid=true
)
savefig(p2, plotsdir(script_name, "parametric_scan_cutter_comparison

```

График метрики scale_ratio по n (раздельно для case) (scale_ratio = r_cutter_final / r_boat_max)

```

p3 = plot(size=(900, 520), dpi=150)
for cs in unique(results_df.case)
    sub = results_df[results_df.case .== cs, :]
    plot!(
        p3,
        sub.n, sub.scale_ratio,
        seriestype=:scatter,
        label="case=$cs"
    )
end
plot!(
    p3,
    xlabel="n",
    ylabel="scale_ratio",
    title="XXXXXXXXXXXX scale_ratio X n (XXX XXXXXX case)",
    legend=:topleft,
    grid=true
)

```

```
)
savefig(p3, plotsdir(script_name, "scale_ratio_vs_n.png"))
```

5.11 Бенчмаркинг с разными параметрами

```
println("\n" * "="^60)
println("XXXXXXXXXXXX XXX XXXXXXX n (XXX case)")
println("="^60)

benchmark_results = []
```

Возьмём сетку n из param_grid и оба case (как в сканировании)

```
for n_value in param_grid[:n], case_value in param_grid[:case]
    bench_params = Dict(
        :n => n_value,
        :s => base_params[:s],
        :fi => base_params[:fi],
        :case => case_value,
        :Xspan => base_params[:Xspan],
        :nX => base_params[:nX],
        :tspan_boat => base_params[:tspan_boat],
        :nt_boat => base_params[:nt_boat],
        :solver => base_params[:solver]
    )

function benchmark_run()
```

Только катер (ODE) — это и есть «вычислительная часть»

```

        p = (n=bench_params[:n], s=bench_params[:s], fi=bench_params[:fi])
        r0 = bench_params[:case] == :plus ? bench_params[:s]/(bench_params[:span]) : 0
        prob = ODEProblem(cutter_ode!, [r0], bench_params[:span], p)
        return solve(prob, bench_params[:solver]; saveat=LinRange(bench_params[:span], 0, bench_params[:span], bench_params[:span]))
    end

    println("\nXXXXXXXXXX XXXX n = $n_value, case = $case_value:")
    b = @benchmark $benchmark_run() samples=80 evals=1
    tsec = median(b).time / 1e9
    println("XXXXXXXXXX XXXXX: ", round(tsec; digits=4), " XXXX")

    push!(benchmark_results, (n=n_value, case=string(case_value), t=tsec))
end

bench_df = DataFrame(benchmark_results)

```

График времени вычисления от n (раздельно по case)

```

p4 = plot(size=(900, 520), dpi=150)
for cs in unique(bench_df.case)
    sub = bench_df[bench_df.case .== cs, :]
    plot!(
        p4,
        sub.n, sub.time,
        seriestype=:scatter,
        label="case=$cs"
    )
end
plot!(

```

```

    p4,
    xlabel="n",
    ylabel="Среднее время вычисления, сек",
    title="Среднее время вычисления ODE для n (разные случаи)",
    legend=:topleft,
    grid=true
)
savefig(p4, plotsdir(script_name, "computation_time_vs_n.png"))

```

5.12 Сохранение всех результатов

```

@save datadir(script_name, "all_results.jld2") base_params base_params
@save datadir(script_name, "all_plots.jld2") p1 p1b p2 p3 p4

println("\n" * "="^60)
println("Содержимое папки")
println("="^60)
println("\nСодержимое папки:")
println(" • data/$(script_name)/single/ - исходные данные")
println(" • data/$(script_name)/parametric_scan/ - результаты параметрического сканирования")
println(" • data/$(script_name)/all_results.jld2 - все результаты")
println(" • plots/$(script_name)/ - все графики")
println(" • data/$(script_name)/all_plots.jld2 - все графики")
println("\nСодержимое папки:")
println(" using JLD2, DataFrames")
println(" @load \"data/$(script_name)/all_results.jld2\"")
println(" println(results_df)")

```

5.13 Анализ результатов моделирования

В ходе работы было проведено численное исследование движения катера, описываемого дифференциальным уравнением

$$\frac{dr}{d\theta} = \frac{r}{\sqrt{n^2 - 1}},$$

а также выполнено сравнение его траектории с траекторией лодки, заданной аналитически и переведённой в полярные координаты.

5.14 Базовые эксперименты

5.14.1 1. Случай (case = plus)



Рисунок 5.1: Базовый эксперимент (case=plus)

На полярном графике видно, что траектория катера имеет вид расходящейся спирали. Радиус монотонно увеличивается при росте угла φ , причём скорость увеличения возрастает по мере удаления от центра. Это объясняется тем, что производная $dr/d\varphi$ пропорциональна самому радиусу r , следовательно, рост носит экспоненциальный характер по углу.

Траектория лодки представляет собой луч, соответствующий прямолинейному движению в декартовой системе координат. В полярной системе она выражается линейной зависимостью радиуса от параметра движения.

Катер постепенно удаляется от центра быстрее, чем лодка, что видно по увеличивающемуся расстоянию между кривыми.

5.14.2 2. Случай (case = minus)



Рисунок 5.2: Базовый эксперимент (case=minus)

Во втором базовом эксперименте начальный радиус больше, поэтому катер стартует дальше от полюса. Спираль сохраняет тот же характер роста, однако вся траектория масштабирована наружу.

Различие между режимами $\text{case}=\text{plus}$ и $\text{case}=\text{minus}$ проявляется именно в начальном положении и общем масштабе кривой, тогда как форма роста остаётся одинаковой.

5.15 Параметрическое сканирование по n

Сканирование: траектории катера (ODE) при разных n и case

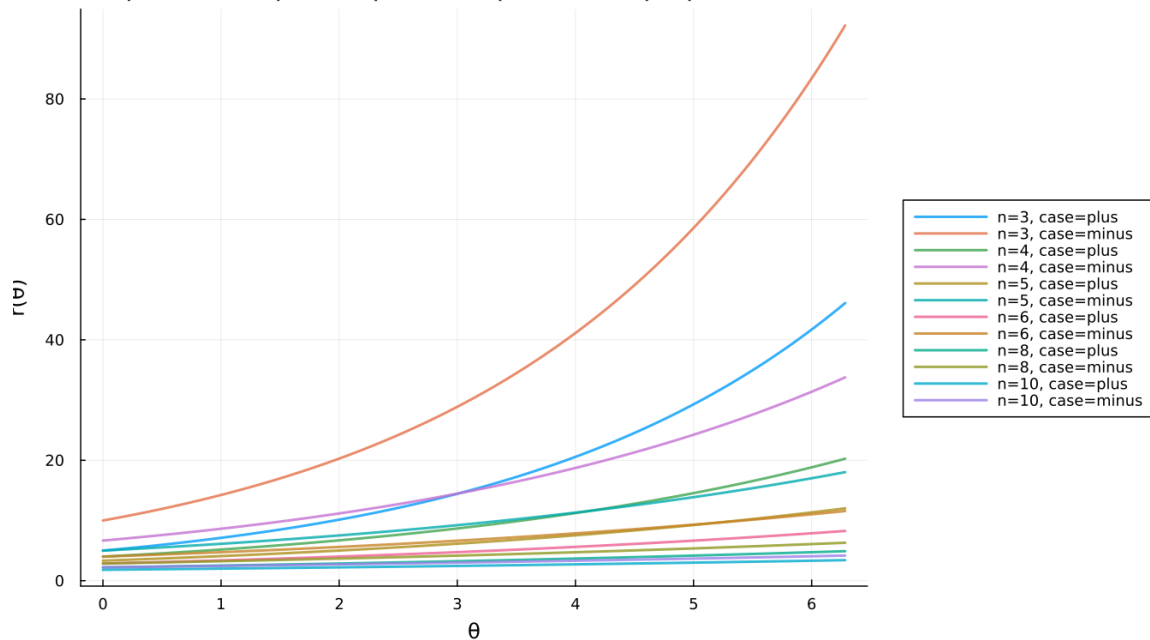


Рисунок 5.3: Сканирование траекторий катера

Было исследовано влияние параметра n на динамику системы для обоих режимов начального условия.

Из уравнения следует, что коэффициент роста равен

$$1/\sqrt{n^2 - 1}$$

. При увеличении n этот коэффициент уменьшается, следовательно:

- при малых n спираль расходится быстрее;
- при больших n рост радиуса замедляется;
- траектории становятся более «пологими».

На графике чётко видно, что для $n=3$ наблюдается наиболее быстрый рост, а для $n=10$ — самый медленный.

5.16 Анализ метрики scale_ratio

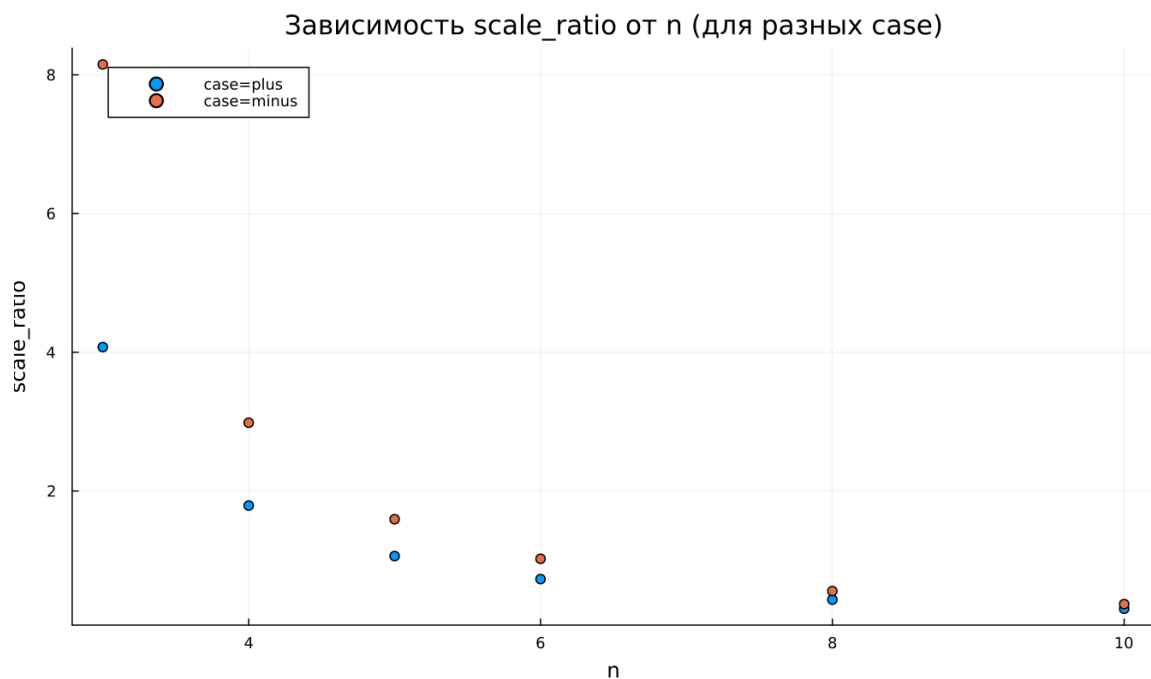


Рисунок 5.4: Зависимость scale_ratio от n

Введённая метрика

$$\text{scale_ratio} = \frac{r_{\text{final}}}{\max(r_{\text{boat}})}$$

характеризует относительный масштаб траектории катера по сравнению с лодкой.

График показывает:

- при малых n значение метрики значительно больше единицы — катер существенно опережает лодку по радиальному росту;
- при увеличении n значение резко уменьшается;
- для больших n траектории становятся сопоставимыми по масштабу.

Для режима `case=minus` значения метрики выше, поскольку начальный радиус больше.

5.17 Время вычислений

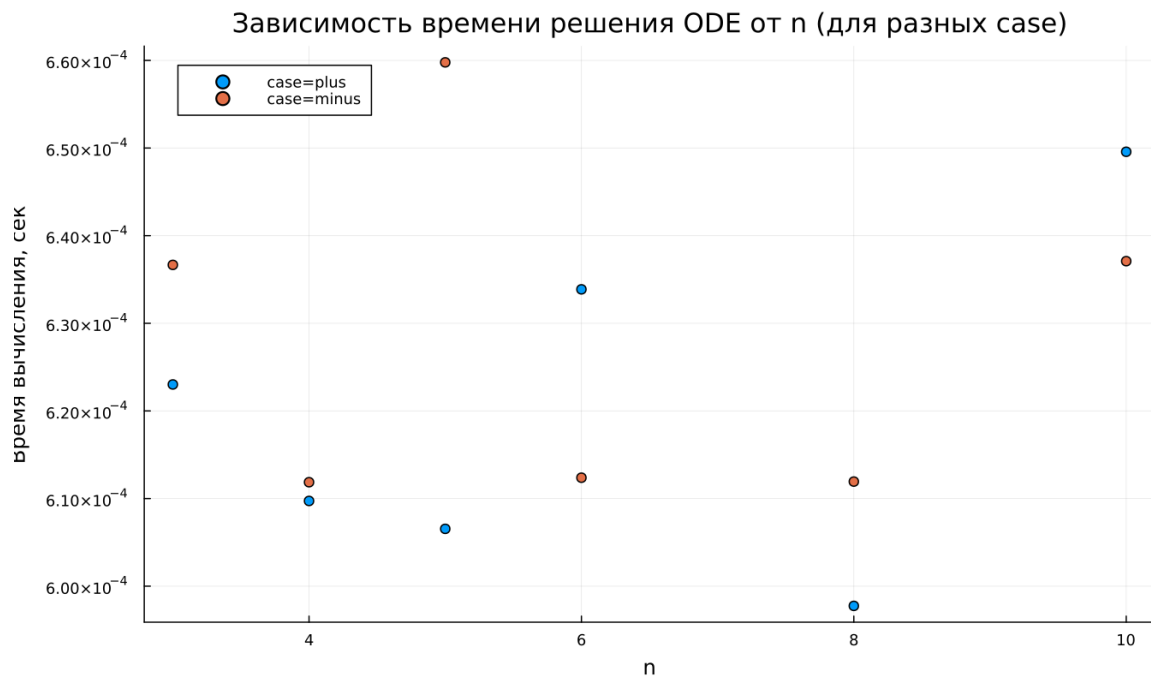


Рисунок 5.5: Зависимость времени вычисления от n

Был выполнен бенчмаркинг решения ОДУ при различных значениях n .

Из графика видно:

- время вычисления находится в пределах $\sim 6 \times 10^{-4}$ секунды;
- существенной зависимости от параметра n не наблюдается;

- небольшие колебания связаны с особенностями адаптивного шага интегрирования и численной обработки.

6 Выводы

1. Траектория катера в полярных координатах представляет собой экспоненциально расходящуюся спираль.
2. Параметр n управляет скоростью радиального роста: чем больше n , тем медленнее увеличивается радиус.
3. Начальное условие (case) определяет масштаб траектории, но не меняет её качественный характер.
4. Численная устойчивость метода высокая, а вычислительные затраты практически не зависят от параметров модели.

Полученные результаты полностью согласуются с аналитической структурой исходного дифференциального уравнения.

Список литературы

1. Задача о погоне